HONEYWELL SOFTWARE ENGINEERING TEAMS — IA · BA · PA&T

AI Champions Programme — GitHub Copilot Enterprise, SDD, MCP & Agentic Software Engineering

Module 11

Observability, ROI & Governance

See what your agents actually do, understand where the tokens go, prove whether the work is worth it, and put governance around it — the practical monitoring layer that connects AI-assisted engineering to quality, cost and value.

DURATION
1.5 hours

FORMAT
Concepts · analysis lab

PRIMARY TOOL
Trace viewer + ROI worksheet

How We'll Spend These Ninety Minutes

Four questions in order: what happened, what did it cost, was it worth it, who is accountable.

01

Observability & Traces

15 min

Traces, spans, replay and the failure patterns they reveal.

02

Token Economics

15 min

Where tokens are wasted and the levers — including a self-hosted model — that reduce them.

03

ROI & Business Value

15 min

Cost per accepted output, and a compact before/after estimate.

04

Governance & Leadership Reporting

10 min

Scale, tune, restrict or retire — and how to report it.

05

Lab: Trace, Cost, Decide

30 min

Inspect traces, find cost leaks, build an ROI estimate and make a decision.

06

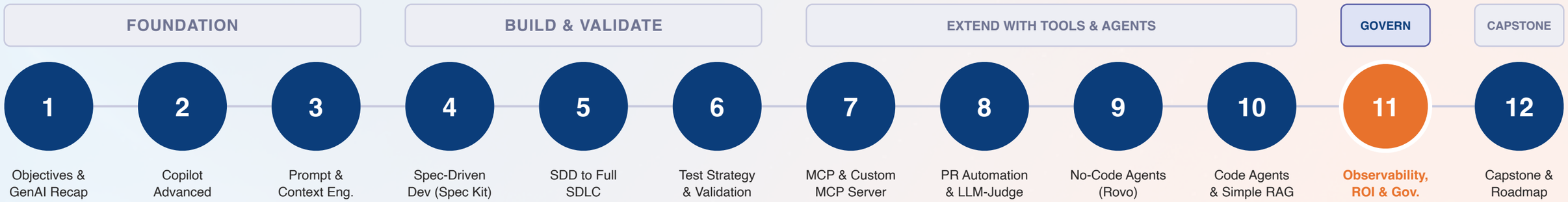
Best Practices & Wrap-up

5 min

Do and avoid, then the capstone handoff in Module 12.

Module 11 in the 12-Module Journey

The agents are built and governed in small pieces; now we measure the whole.



M07

MCP tools and servers are spans you can trace and govern

M08

Judge scores and review outcomes are quality signals

M09–10

The studio and code agents are what you now observe

M12

Your capstone adoption plan needs cost, value and risk evidence

Why this matters: adoption without measurement is a belief, not a result. This module gives you the evidence leadership will ask for.

Four Questions, in Order

Each answer unlocks the next. Skip one and the next is guesswork.

OBSERVABILITY

What did the agent do?

Traces, spans, replay, failures

TOKEN ECONOMICS

What did it cost?

Tokens per run, leaks, levers

ROI & VALUE

Was it worth it?

Cost per accepted output

GOVERNANCE

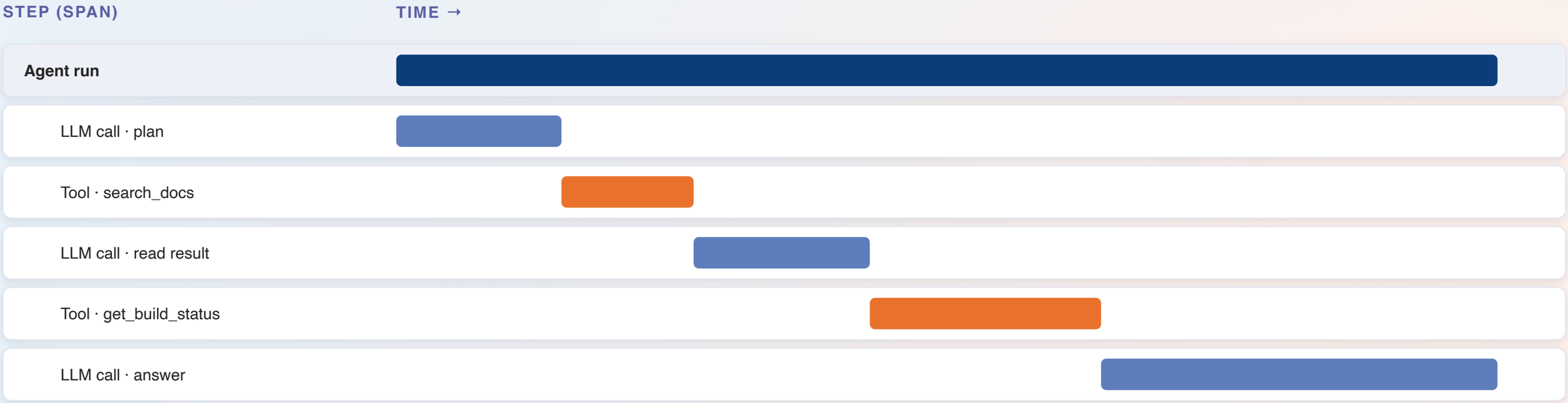
Who decides, and who answers?

Scale, tune, restrict, retire

The through-line: the same trace data feeds all four answers. Collect it once, early, and every later question gets cheaper to answer.

Observability: Seeing an Agent Run as a Trace

A trace is the agent's run as a tree of timed steps, called spans.



EACH SPAN CAN CARRY

- Duration
- Tokens in / out
- Cost
- Input & output
- Status / error

Illustrative shape, not real data: traces use the OpenTelemetry model, so spans from your SDK agent, tools and MCP servers can sit in one view.

Trace Viewers: Four Ways to Look at the Same Trace

A good trace viewer turns OpenTelemetry trace data into views for debugging agents.

Tree view

Parent–child hierarchy of steps; highlights problem patterns and collapses repeats

Timeline view

Gantt-style flow showing bottlenecks, concurrency and cost building up

Details panel

Each step's input, output, cost and performance attributes

Sequence replay

Step-by-step playback to follow decisions and spot circular logic

HOW IT FITS

Agent emits OpenTelemetry spans → **Converter maps them to a UI schema** → **Components render the views**

In the lab: you use the programme's trace sandbox with sample runs. In your own projects, the first step is making the agent emit traces at all.

Reading a Trace: Failure Patterns to Look For

Most problems have a visible shape before you read a single prompt.

PATTERN	WHAT YOU SEE	LIKELY CAUSE
Loop	The same tool called again and again	Missing stop condition or unclear instruction
Retry storm	Repeated failures with growing context	Bad tool error or no circuit breaker
Wrong tool	A tool chosen that cannot answer	Vague tool descriptions or too many tools
Empty retrieval	Search returns nothing, the agent answers anyway	Missing content or weak “not found” rule
Bloated payload	One step swallows most of the tokens	Raw tool output pasted into context

Real-world use: when a build-failure agent from Module 10 felt slow and costly, the trace is where you find which of these five it was.

Quality and Control Signals: What to Track per Workflow

Six signals tell you whether an agent is useful, safe and affordable. Define them before you collect them.

Accepted output rate

Share of outputs a human accepts as-is or with light edits

Override & edit rate

How often people change or reject the agent's result

Failure & retry rate

Runs that error, loop or need re-running

Latency per run

Time from request to usable answer

Tokens per run

Input and output tokens, split by step

Cost per accepted output

Total cost divided by outputs that were accepted

Careful with proxies: tokens and speed are easy to count; acceptance and rework are what show real value. Track both kinds, and set thresholds from your own baseline.

Token Economics: Where Agent Tokens Quietly Go

Agents resend context on every step, so small habits compound.

LEAK	WHAT HAPPENS	FIRST FIX
Context accumulation	Full history is resent at every call	Summarise older turns; reuse a stable prompt prefix with caching
Unbounded retries	Failed attempts and error traces stay in context	Circuit breaker; prune failed attempts; short deterministic error notes
Unfiltered tool payloads	Raw API responses pasted into the prompt	Extract only the fields the agent needs before it sees them
One big model for everything	Costly model on trivial steps	Route simple steps to smaller models (watch routing overhead)
Giant static prompt	Every tool and edge case on every call	Inject only the tools and rules the current step needs

Find it before fixing it: use the trace to see which step carries the tokens. Optimising a step that costs little is effort wasted.

Cost and Control Levers, Including a Self-Hosted Model

Each lever trades something. Pick the cheapest one that solves the problem you measured.

Trim context

Send less per call

+ Less cost, faster

– May drop needed detail

Cache & reuse

Reuse stable prompt prefixes

+ Pay once for repeated text

– Needs stable prompts

Route by task

Smaller model for simple steps

+ Lower cost per step

– Routing adds complexity

Self-hosted SLM / LLM

Run a model in your own environment

+ More control over data and cost shape

– You own ops, updates and quality

About the demo: the self-hosted SLM demo is a lever to discuss, not a recommendation. Whether it pays off depends on volume, quality needs and who runs it.

ROI: Cost per Accepted, Production-Quality Output

Count what was accepted, not what was generated — and include the humans.

Total cost
tokens + tooling + human review time

÷

Accepted outputs
kept at production quality

=

Cost per accepted output
compare before vs after

A COMPACT BEFORE / AFTER WORKSHEET

INPUT	WHERE IT COMES FROM	WATCH OUT FOR
Runs and accepted outputs	Traces plus reviewer decisions	Counting generated output as accepted
Token and tool cost per run	Trace usage data, billing export	Ignoring failed and retried runs
Human review and edit time	Short time sampling, reviewer notes	Treating review as free
Time saved vs the manual way	A measured baseline, not a guess	Using an optimistic estimate
Rework and later defects	Bug and revert records	Missing costs that arrive weeks later

Be honest about uncertainty: an estimate built from measured inputs and stated assumptions beats a confident number nobody can trace.

Decide: Scale, Tune, Restrict or Retire

Every workflow ends up in one of four places. The evidence decides which.

Scale

WHEN

Accepted rate high, cost per output acceptable, few incidents

EXAMPLE (ILLUSTRATIVE)

Build-failure explainer used by more teams

Tune

WHEN

Useful but costly or inconsistent; leaks visible in traces

EXAMPLE (ILLUSTRATIVE)

Release-notes drafter with a bloated tool payload

Restrict

WHEN

Valuable but risk shows; needs tighter permissions or review

EXAMPLE (ILLUSTRATIVE)

Triage agent with write access limited to drafts

Retire

WHEN

Low acceptance or cost exceeds value; no owner

EXAMPLE (ILLUSTRATIVE)

Runbook helper reading stale documents

Retire is a success too: stopping a workflow that does not pay off frees budget and attention for the ones that do. Make it a normal outcome, not a failure.

AI Governance: Four Functions You Already Started

The NIST AI Risk Management Framework (voluntary) groups governance into Govern, Map, Measure and Manage.

Govern

Named owners, policies, an approval path for new use cases

ALREADY IN THIS PROGRAMME

Governance record (M07), ownership (M09)

Map

Inventory systems, data, tools, people affected, ways to fail

ALREADY IN THIS PROGRAMME

Server and agent inventory; handover pack

Measure

Metrics tied to failure modes; tracked over time

ALREADY IN THIS PROGRAMME

Traces, judge scores, override rate

Manage

Triggers, incident handling, change control, a pause switch

ALREADY IN THIS PROGRAMME

Escalation rules (M08); off switch

Keep it proportionate: the framework is voluntary and no one is “certified” against it. Use it as a checklist for gaps, scaled to the risk of each workflow.

Leadership Reporting: One Page That Drives a Decision

Fewer, better-defined measures. Every tile has an owner and a next step.

Adoption

Who uses which workflows, how often

Metric · trend · owner

Quality

Accepted rate, override rate, defects found later

Metric · trend · owner

Cost

Token and tool cost, cost per accepted output

Metric · trend · owner

Value

Time saved vs measured baseline, rework avoided

Metric · trend · owner

Risk & incidents

Policy exceptions, incidents, open findings

Metric · trend · owner

Decisions needed

Scale, tune, restrict or retire — with evidence

Metric · trend · owner

Report trends and decisions, not dashboards: a number with no owner and no decision attached is decoration. Agree the cadence and thresholds with the people who will act on them.

Lab: Trace, Cost, Decide

LAB 1 · 30 min

Find the leaks in sample traces, estimate the ROI, and make a call on one workflow.

Goal Inspect sample traces, identify token and cost leakage, build a before/after ROI estimate, and decide scale, tune, restrict or retire.

STEPS

1. Open the sample traces. Find three problems (for example a loop, a bloated payload, repeated context) and note the step and evidence for each.
2. Propose one fix per problem. Using the sample telemetry, work out before/after tokens per run and accepted outputs.
3. Fill the ROI worksheet with the sample inputs and compute cost per accepted output before and after. State your assumptions.
4. Watch the self-hosted SLM demo, then choose scale, tune, restrict or retire for one workflow and write down your reasons.

FILES YOU CREATE

repo/

```
lab11/
  trace-findings.md
  before-after.md
  roi-worksheet.md
  decision.md
```

DONE WHEN

Each finding cites a step in a trace, the ROI sheet shows its assumptions, and decision.md gives a clear call with evidence.

Best Practices: Observe, Measure, Govern

Measure early, count accepted work, and keep a named owner for every workflow.

DO THIS	AVOID THIS
Make every agent emit traces from day one	Judge an agent from a few good-looking runs
Find the costly step in the trace before optimising	Count generated output as if it were accepted
Measure cost per accepted output, including human time	Report token savings while ignoring rework
Set baselines before you claim improvement	Quote sample or vendor numbers as your own results
Give every workflow an owner and a scale/tune/restrict/retire review	Let a workflow run with no owner or review date
Report trends and decisions on one page	Treat retiring a workflow as a failure

Golden rule: if you cannot trace what an agent did, price what it cost and show what it was worth, you cannot responsibly scale it.

What You Can Do Now

Six outcomes — from reading a trace to making a governed decision.

- ✓ Read agent traces and recognise common failure patterns
- ✓ Choose quality and control signals for a workflow
- ✓ Find where tokens leak and pick the right lever
- ✓ Estimate cost per accepted output with stated assumptions
- ✓ Decide to scale, tune, restrict or retire with evidence
- ✓ Map your controls to Govern, Map, Measure and Manage

NEXT: MODULE 12 — HONEYWELL CAPSTONE & ROADMAP · 2 HOURS

Bring the whole journey together in one adoption plan, with evidence on quality, cost and risk.

TODAY'S PATH

Observe → Cost → Value → Decide → Govern

Questions & Discussion

Module 11 — Observability, ROI & Governance